Count Squares

AlgoExpert · Hard · Arrays

The problem

- Number of squares (any orientation) whose four corners are all in the point set.
- For each pair as a diagonal, compute the other two corners; each square is found twice.

Intuition

- Any two points can be a square's diagonal; the other two corners are then fixed, so just check they exist.

Approach

- 1. Put all points in a hash set
- 2. For each pair, compute the two remaining corners by rotating around the midpoint
- 3. Count when both exist
- 4. Divide by two since each square has two diagonals

Java solution

```java
public class CountSquares {
    public static int countSquares(int[][] points) {
        Set<String> set = new HashSet<>();
        for (int[] p : points) set.add(p[0] * 2 + "," + p[1] * 2);
        int count = 0;
        for (int i = 0; i < points.length; i++)
            for (int j = 0; j < points.length; j++) {
                if (i == j) continue;
                long x1 = points[i][0] * 2L, y1 = points[i][1] * 2L, x2 = points[j][0] * 2L, y2 = points[j][1] * 2L;
                long mx = (x1 + x2) / 2, my = (y1 + y2) / 2, dx = x1 - mx, dy = y1 - my;
                if (set.contains((mx - dy) + "," + (my + dx)) && set.contains((mx + dy) + "," + (my - dx))) count++;
            }
        return count / 4;
    }
}
```

Complexity

- Time: O(n^2)
- Space: O(n)

Recap

- Any two points can be a square's diagonal; the other two corners are then fixed, so just check they exist.
- Complexity: O(n^2)